

CNV Project Report

Pedro Pereira (ist98962) Guilherme Campos (ist1106909) João Ribeiro (ist1107251)
Group 28, Instituto Superior Técnico

1 INTRODUCTION

The objective of this project was to deploy the Nature@Cloud workloads on AWS while providing efficient request scheduling, dynamic resource management, and execution cost awareness. The workloads exhibit highly heterogeneous execution behavior, meaning that requests targeting the same endpoint may differ significantly in computational cost depending on their input parameters. Traditional load balancing approaches based solely on request counts or CPU utilization are therefore insufficient, as they cannot distinguish lightweight requests from computationally expensive ones.

To address this challenge, we developed a distributed execution platform centered around runtime instrumentation, complexity estimation, cost-aware scheduling, and dynamic scaling. The system combines a custom Java load balancer, a custom auto-scaler, a worker pool running the Nature@Cloud services, a metrics collection subsystem based on Javassist instrumentation, DynamoDB for persistent storage of execution statistics, and AWS Lambda for opportunistic execution of small requests during periods of high worker utilization.

During the checkpoint phase, the project relied primarily on AWS-managed Elastic Load Balancing and Auto Scaling mechanisms while exploring complexity models and metric collection. The final solution evolved considerably from that design. Rather than delegating scheduling decisions entirely to AWS services, request routing is now performed by a custom load balancer that estimates execution cost before dispatching requests. This allows the scheduler to reason about computational workload instead of simply balancing the number of requests. More on that in future sections.

A major refinement in the final design is the handling of cache misses in historical cost estimation. While the checkpoint design relied on querying DynamoDB whenever no local cache entry existed, this approach was found to introduce unnecessary latency and external dependency overhead. The final system instead uses cost estimation immediately on cache misses and relies on a periodic background refresher (every 60 seconds) to populate relevant entries asynchronously. This design significantly reduces request-time latency while preserving eventual consistency of cost data.

The resulting system combines online execution metrics with historical workload information to make scheduling and scaling decisions. The architecture seeks to maximize throughput, reduce load imbalance between workers, and improve responsiveness under varying workload conditions while maintaining a lightweight runtime overhead.

2 SYSTEM ARCHITECTURE

Figure 1 presents the final architecture of the system. The final solution consists of four major components:

- **Load Balancer:** A Java-based HTTP server responsible for request parsing, cost estimation, scheduling, and Lambda offloading.
- **Worker Pool:** A set of EC2 instances executing the Nature@Cloud workloads.
- **Metrics Storage:** DynamoDB tables storing historical execution metrics grouped by workload type.
- **Auto-Scaler:** A monitoring service responsible for provisioning and terminating workers according to estimated workload.

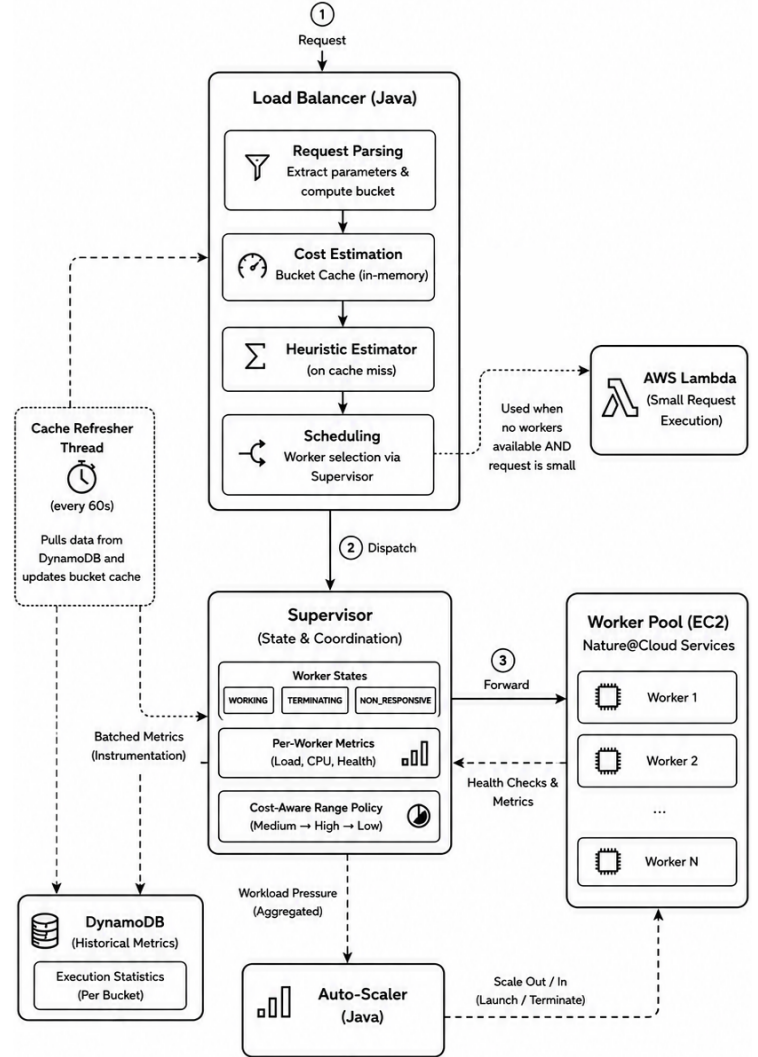


Fig. 1: System architecture overview

The system is coordinated by a Supervisor component that maintains the state of all worker instances. It continuously monitors worker health through periodic health checks, classifies workers into operational pools (WORKING, TERMINATING, NON_RESPONSIVE), and maintains per-worker load and CPU usage statistics. The Load Balancer relies on the Supervisor as the source of truth for worker availability and scheduling decisions.

2.1 Request Handling

The request flow begins when a client issues a request to one of the workload endpoints. The load balancer extracts the workload parameters and computes a bucket representation that identifies similar historical executions.

The request is then processed through a multi-stage cost estimation pipeline. The scheduler first attempts to retrieve an estimated execution cost from an in-memory cache, a fast path. If no cached value exists, system estimates cost using heuristics (no DynamoDB access). A periodic background thread is responsible for refreshing the cache by populating it asynchronously from DynamoDB, increasing future cache hits.

2.2 Worker Selection

After estimating the request cost, the scheduler delegates worker selection to the Supervisor. The Supervisor first queries the WORKING pool and, if needed, the TERMINATING pool. Within each

pool, workers are selected using a cost-aware range policy that prioritizes medium-load workers first, then high-load workers, and finally low-load workers, ensuring requests are placed on workers that can best absorb additional load without immediate saturation.

Compared with the checkpoint architecture, the final design moves scheduling decisions from AWS-managed services into the application layer. During the checkpoint phase, request distribution relied primarily on Elastic Load Balancing while Auto Scaling decisions were based on infrastructure metrics. That approach had no visibility into request complexity and therefore treated all requests equally. The final architecture introduces workload awareness by incorporating historical execution information into both scheduling and scaling decisions. This enables more balanced resource utilization and reduces situations where a worker becomes overloaded by a small number of computationally expensive requests.

3 JAVASSIST INSTRUMENTATION

The system relies on runtime instrumentation performed via Javassist to collect fine-grained execution statistics for each request. These metrics are emitted at the end of request execution and represent structural and runtime properties of the workloads. The goal of instrumentation is not only to measure performance, but to provide meaningful signals that correlate with computational cost across heterogeneous inputs. Each workload exposes a different set of parameters that influence its computational behavior. These parameters are extracted at the load balancer level and also recorded alongside runtime metrics for later aggregation.

For the fractals workload, complexity is primarily driven by the image resolution and the number of iterations. Specifically, the relevant parameters are:

- `w, h` (combined into a derived resolution value)
- `iterations`

These parameters directly influence the number of pixel-level computations performed by the algorithm, with higher resolutions and iteration counts increasing execution cost.

For the DNA workload, complexity depends on the input sequence size. The relevant parameters are:

- `seq1`
- `seq2`

These are converted into a single effective sequence length by summing the lengths of both sequences. This derived value is used as the primary driver of computational cost, as alignment complexity scales with sequence size.

For the Gray-Scott workload, execution cost is influenced by both spatial and temporal parameters:

- `size`
- `maxIterations`

These parameters determine the size of the simulation grid and the number of evolution steps. Unlike the other workloads, Gray-Scott also exposes additional internal characteristics such as method-level execution counts, which are captured by instrumentation but are not directly used as primary input parameters for bucketing. Empirical testing showed that Gray-Scott control parameters such as `f`, `k`, and the initial seed configuration (center, ring, stripe) had negligible impact on overall computational cost compared to grid size and iteration count.

3.1 Collected Runtime Metrics

Each request produces the following instrumentation outputs:

- **Instructions:** total executed bytecode-level instructions.
- **Blocks:** basic block execution count.
- **Methods:** number of method invocations during execution.
- **Constructors:** number of constructor calls.
- **Fragmentation:** a structural metric representing execution dispersion across code regions.

Instruction count provides a direct approximation of CPU effort, as it reflects the total amount of low-level work performed by the program. Method invocations capture dispatch overhead and call-chain depth, relevant in workloads where computation is distributed across multiple components. Constructor counts represent object allocation pressure, that influence memory allocation and garbage collection activity. Fragmentation reflects how execution is distributed across the program structure, with highly fragmented execution indicating frequent transitions between distinct code regions and increased control-flow complexity. Block counts provide additional granularity on execution structure.

Across workloads, we observe distinct execution profiles. DNA and Fractals exhibit relatively low stable method counts, whereas Gray-Scott shows significantly higher method-level activity due to its iterative simulation structure. This is visible in observed traces, where Gray-Scott requests present orders of magnitude more method invocations compared to the other workloads. In particular, Gray-Scott's high method invocation rate and fragmentation values indicate significantly higher runtime complexity, despite sometimes similar instruction magnitudes compared to other workloads.

Figure 2 shows how each workload parameter affects complexity, approximated as instruction count for simplicity.

3.2 Role of Instrumentation in the System

The collected metrics serve two purposes. First, they are used to derive an offline cost model that approximates request complexity. Second, they are stored in DynamoDB and later aggregated into bucketed representations of input parameter space.

During the checkpoint phase, the system relied heavily on querying historical raw records from DynamoDB for similarity matching. In the final design, this approach was replaced by a bucketed abstraction combined with in-memory caching and periodic refresh. This reduces database dependency while maintaining sufficient resolution to distinguish between different cost regimes.

Instrumentation overhead is minimized by performing aggregation at the end of request execution and asynchronously buffering results before batch insertion into DynamoDB. More specifically, a background flusher thread periodically batches and writes metrics every 20 seconds to DynamoDB's workload tables. This ensures that metric collection does not interfere with request latency.

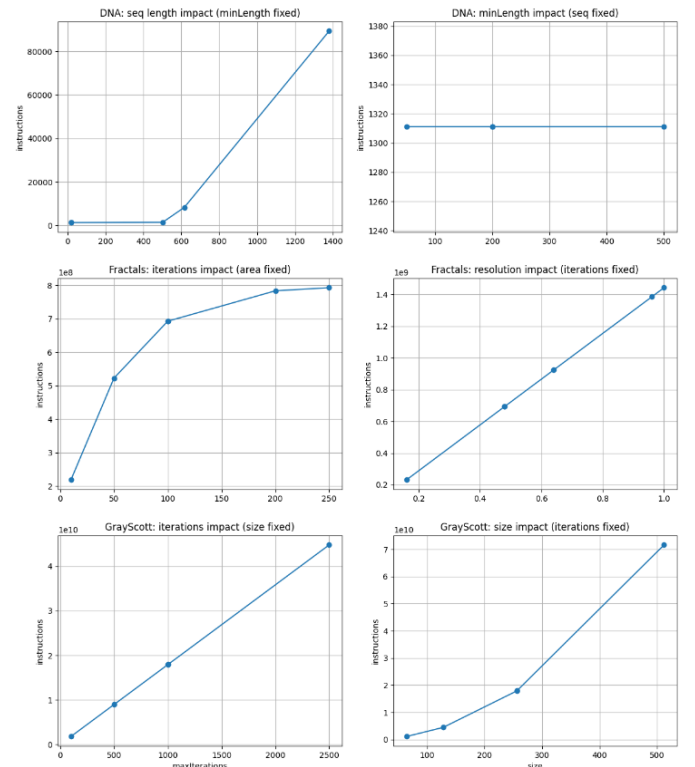


Fig. 2: Parameter complexity distribution

The system combines a compact parameter bucketing scheme with a hierarchical caching strategy that minimizes external storage access while preserving generalization across similar requests.

4.1 Bucketed Metrics Representation

Request parameters are transformed into bucket keys using fixed bucket sizes (e.g., 40-unit intervals). This reduces dimensionality and ensures that similar requests are grouped. This abstraction enables generalization across similar workloads while limiting DynamoDB storage growth and improving cache hit rate.

$$\text{bucket}(x) = \left\lfloor \frac{x}{b} \right\rfloor \cdot b$$

For example, with a bucket size $b = 40$:

- A request with $x = 83$ is mapped to $\text{bucket}(83) = \lfloor 83/40 \rfloor \cdot 40 = 80$
- A request with $x = 97$ is also mapped to $\text{bucket}(97) = \lfloor 97/40 \rfloor \cdot 40 = 80$

Thus, both requests are grouped into the same (80) bucket, even though their raw parameter values slightly differ.

4.2 Cache Hierarchy

The system uses a two-level cache:

- **MetricsCache (L1):** In-memory concurrent hash map storing bucket-to-cost mappings.
- **DynamoDB (L2):** Persistent storage for historical execution costs, also keyed using buckets for compatability.

A key design decision in the final system is that DynamoDB is never queried synchronously on cache misses. Instead, the system immediately falls back to heuristic cost estimation and relies on a background refresher thread to asynchronously populate missing entries. This avoids blocking request execution on external network latency.

5 REQUEST COST ESTIMATION

The current complexity model is multiplicative and defined as:

$$\text{Cost} = \left(\frac{\text{inst}}{10^6} \right) \times (1 + 0.05 \cdot \text{ct}) \times (1 + \log(1 + fg))$$

Where:

- inst = total executed bytecode instructions
- ct = constructor invocations
- fg = fragmentation metric from basic block size variance

This model captures three key dimensions:

- **Base computation cost:** proportional to instruction count.
- **Structural overhead:** constructors approximate object creation overhead.
- **Control complexity:** fragmentation increases branching cost logarithmically.

Compared to the checkpoint version, the final complexity model removes the method invocation term. Experimental observations showed that method counts remained relatively stable for DNA and Fractals requests, while varying significantly only in Gray-Scott executions due to the workload's iterative simulation structure. As a result, method invocations were not a consistent predictor of complexity across all workloads and introducing them into a unified cost model would disproportionately penalize Gray-Scott requests. Since the objective of the model is to provide a workload-independent approximation of execution cost, the final version retains only metrics that exhibited meaningful correlation across all workloads.

Alternative approaches considered included linear regression over historical samples and reinforcement-based cost learning. However, these were rejected due to increased complexity and runtime overhead.

6.1 Worker Selection Strategy

Worker selection is delegated to the Supervisor component, which continuously tracks worker state, CPU usage and estimated outstanding workload. Workers are classified into three pools:

- **WORKING:** healthy workers available for request execution.
- **TERMINATING:** workers scheduled for removal but still capable of completing or receiving requests if necessary.
- **NON_RESPONSEIVE:** workers that temporarily failed health checks and are undergoing recovery validation.

Unlike traditional load balancers that rely solely on request counts, the scheduler incorporates the estimated computational cost of each request. When a request arrives, its cost is estimated using the mechanisms described in the previous section and this value is treated as the expected load contribution of the request.

Each worker maintains an estimate of its current accumulated load, computed as the sum of the costs of all requests currently executing on that worker. This allows the scheduler to reason about future workload rather than only current CPU utilization. Worker selection follows a hierarchical policy. The Supervisor first searches the WORKING pool. If no suitable worker exists, the TERMINATING pool is considered before falling back to Lambda. Within each pool, workers are partitioned according to CPU usage: first workers in the optimal utilization range (30% – 80% CPU), workers under high utilization (80% – 100% CPU) and finally workers under low utilization (below 30% CPU).

The scheduler first attempts to place requests on workers in the optimal range, avoiding both overloaded workers and nearly idle workers. By prioritizing moderately loaded instances, the system preserves spare capacity for future expensive requests while preventing unnecessary concentration of work on lightly loaded machines. Within each utilization range, workers are ordered by estimated load and the highest-load worker that can still accommodate the incoming request without exceeding the maximum load threshold is selected. This strategy promotes load consolidation while avoiding worker saturation. For lightweight requests, if all active workers are already operating above the high-load threshold, requests below a configurable cost threshold are offloaded to AWS Lambda, reducing queueing delays on EC2 workers and reserves VM capacity for expensive requests.

Algorithm 1 Worker Selection

```

1:  $\text{cost} \leftarrow \text{estimateRequestCost}(\text{request})$ 
2: if  $\text{cost} \leq \lambda_{\text{threshold}}$  and all workers have CPU > 80% then
3:   Execute request in AWS Lambda
4:   return
5: end if
6: for pool in {WORKING, TERMINATING} do
7:   for  $\text{cpuRange}$  in {[30,80], [80,100], [0,30]} do
8:     Find highest-load worker in  $\text{cpuRange}$ 
9:     satisfying:

$$\text{workerLoad} + \text{cost} < \text{MAX\_LOAD}$$

10:    if worker exists then
11:      Assign request to worker
12:      return
13:    end if
14:   end for
15: end for
16: if  $\text{cost} \leq \text{LAMBDA\_MAX\_COST}$  then
17:   Execute request in AWS Lambda
18:   return
19: end if
20: Request will be retried upon worker availability

```

6.2 Lambda Offloading Policy

Lambda is used when:

- Estimated cost is below a predefined threshold, and
- All EC2 workers are operating under high load, or
- No suitable EC2 worker can accommodate the request within its remaining capacity, while the request remains within Lambda execution limits

This design introduces a fast fallback execution path that avoids blocking lightweight requests behind compute-intensive workloads. By isolating small or latency-sensitive requests from EC2 queue buildup, the system reduces tail latency and prevents head-of-line blocking. At the same time, Lambda is not used as a primary execution target but as a bounded overflow mechanism, preserving EC2 resources for sustained high-cost workloads.

7 AUTO-SCALING ALGORITHM

The Auto-Scaling component is responsible for dynamically adjusting the number of EC2 worker instances in response to observed workload pressure. Unlike the checkpoint design, which relied primarily on AWS Auto Scaling Groups reacting to generic infrastructure metrics, the final system introduces a workload-aware scaling strategy driven by request cost estimation and runtime execution signals collected by the Supervisor.

The Auto-Scaler operates as a periodic control loop executed every 10 seconds. At each iteration, it queries the Supervisor for the current system state, including:

- Number of active workers
- Number of non-responsive workers
- Number of pending instance launches
- Average CPU utilization across active workers
- Presence of idle or underutilized workers

7.1 Scaling Policy Overview

The system enforces strict capacity and stability constraints:

- **Maximum cluster size:** 6 instances
- **Minimum cluster size:** 1 instance
- **High CPU threshold:** 0.8
- **Scale-down cooldown:** 60 seconds after the last scale-up event

This ensures controlled elasticity while preventing oscillations under bursty workloads.

7.2 Scale-Up Policy

A scale-up event is triggered when the total number of active, non-responsive, and pending instances is below 6, and the system is under sustained load (high CPU utilization) or operating in an initial empty state. Once triggered, a new EC2 instance is launched using a predefined launch template and immediately marked as "pending" to prevent duplicate provisioning. After the instance reaches the running state and passes health checks, it is registered with the Supervisor and integrated into scheduling decisions.

$$\text{scaleUp} \iff (\text{active} + \text{nonResponsive} + \text{pending} < 6) \wedge (\text{avgCPU} \geq 0.8 \vee \text{active} = 0)$$

7.3 Scale-Down Policy

Scale-down only occurs when the system has more than one active worker, the scale-down cooldown period (60 seconds) has elapsed, and removing one worker does not push the remaining cluster into overload.

$$\text{scaleDown} \iff \frac{\text{avgCPU} \cdot \text{count}}{\text{count} - 1} < 0.8$$

Only idle workers (zero current load) are eligible for removal. This ensures that active computations are never interrupted.

7.4 Instance Lifecycle Management

A key aspect of the final design is explicit handling of instance lifecycle states. Workers are not immediately terminated upon removal decisions. Instead, they are first moved into a *TERMINATING* pool, where:

- They may still complete in-flight requests
- New requests can still be routed if necessary
- Pending requests are re-dispatched by the Supervisor when required

This design guarantees graceful degradation and avoids request loss during scaling transitions. The system also incorporates limited predictive behavior by accounting for pending instance launches and projected post-removal CPU utilization when evaluating scaling decisions.

7.5 Integration with Load Balancing

The Auto-Scaler is tightly coupled with the Load Balancer and Supervisor. Scaling decisions are influenced by:

- CPU utilization trends across workers
- Request cost distribution produced by instrumentation
- Queue pressure inferred from worker load states

Compared to the checkpoint architecture, where scaling was purely reactive and infrastructure-driven (scaling decisions were made with CloudWatch alarms), the final system introduces proactive scaling based on workload semantics and predicted computational demand, and runtime metrics collected directly from the Supervisor, improving responsiveness under heterogeneous workloads. This tight integration allows scaling decisions to react to actual execution pressure rather than delayed infrastructure-level metrics.

Algorithm 2 Auto-Scaling Loop

```

1: while true every 10 seconds do
2:   active  $\leftarrow$  number of WORKING workers
3:   nonResp  $\leftarrow$  number of NON_RESPONSIVE workers
4:   pending  $\leftarrow$  number of pending instances
5:   count  $\leftarrow$  active
6:   avgCPU  $\leftarrow$  average CPU utilization
7:   total  $\leftarrow$  active + nonResp + pending
    $\triangleright$  SCALE UP
8:   if total < 3 then
9:     if avgCPU  $\geq$  0.8 or system is empty then
10:      Launch EC2 instance
11:      Mark instance as PENDING
12:      Wait until instance is RUNNING
13:      Register instance with Supervisor
14:    end if
15:  end if
    $\triangleright$  SCALE DOWN
16:  if count > 1 and timeSinceLastScaleUp > 60s then
17:    if  $\frac{\text{avgCPU} \cdot \text{count}}{\text{count} - 1} < 0.8$  then
18:      idleWorkers  $\leftarrow$  workers with load = 0
19:      if idleWorkers  $\neq \emptyset$  then
20:        Select worker w from idleWorkers
21:        Move w to TERMINATING pool
22:      end if
23:    end if
24:  end if
    $\triangleright$  TERMINATION
25:  for worker in TERMINATION_CANDIDATES sorted by
    low CPU and low load do
26:    Terminate EC2 instance of worker
27:    Remove worker from Supervisor
28:  end for
29: end while

```

When no cached metric exists for a request, the load balancer estimates its cost using a workload-specific heuristic model that maps parameters to an approximate cost based on a linear combination of parameter contributions weighted by empirically calibrated coefficients derived from metrics collected during execution:

$$C(\mathbf{p}) = \sum_{i \in P} w_i \cdot x_i$$

$$\mathbf{p} = \{\text{request parameters}\}, \quad w_i \in R_{\geq 0}, \quad x_i \in N$$

Fractals are highly sensitive to parameter scaling, but their ICount values are overestimated in the complexity model. To correct this, the coefficients were reduced significantly (0.01 for both parameters), normalizing the cost into a lower magnitude range.

$$C_{\text{fractals}}(w, h, i) = 0.01 \cdot i + 0.01 \cdot (w \cdot h)$$

DNA sequence alignment represents a comparatively light workload with near-linear scaling relative to sequence length. As such, a unit coefficient is sufficient.

$$C_{\text{dna}}(L) = 1 \cdot L$$

Gray-Scott is the most computationally intensive workload, exhibiting strong nonlinear growth with respect to grid size and maximum iteration count. Empirical tuning led to higher coefficients (10 and 15 respectively), reflecting its dominant contribution to system load.

$$C_{\text{grayscale}}(s, \text{maxIt}) = 10 \cdot s + 15 \cdot \text{maxIt}$$

To avoid scheduling instability caused by extreme estimates, the final cost is clamped to a predefined maximum threshold. This ensures fairness in worker selection and prevents a single request from monopolizing system capacity due to estimation spikes. Additionally, for Gray-Scott workloads, an optional multiplier of 25 is applied in specific execution modes (e.g., when `stopOnExtinction=false`), further reflecting worst-case computational behavior.

9 FAULT TOLERANCE

Fault tolerance in the final system is achieved through a combination of monitoring, request re-dispatching, and fallback execution paths. The system explicitly tracks in-flight requests and preserves execution continuity across worker failures and scaling events.

9.1 Worker Health Monitoring

The Supervisor continuously monitors all worker instances through periodic health checks. Each worker is queried at fixed intervals, and its state is updated based on HTTP response availability, response correctness (valid OK: CPU format), and measured CPU utilization.

Workers that fail health checks are temporarily moved to a *NON_RESPONSIVE* pool. If recovery succeeds within a grace period, they are reintegrated into the active pool; otherwise, they are marked for termination. This mechanism ensures that transient failures (e.g., network delays or short-lived instance instability) do not immediately trigger removal.

9.2 Graceful Worker Removal and Request Preservation

When a worker is selected for termination, it is not immediately destroyed. Once a worker is first moved to the *TERMINATING* pool, the worker is no longer considered for new request assignments in normal scheduling. However, it is still allowed to complete or expose existing in-flight requests. Its current load

is explicitly extracted before termination. This is implemented through a dedicated draining mechanism in the worker model:

- Each worker maintains a map of active requests (`currentLoad`).
- When termination is triggered, the worker executes a `drainPendingExchanges()` operation.
- This operation atomically extracts all pending `HttpExchange` objects and associated cost metadata, clearing internal state safely under a write lock.

This guarantees that no request is silently dropped during lifecycle transitions.

9.3 Request Re-Dispatch Mechanism

Once pending requests are drained from a terminating worker, they are explicitly re-injected into the load balancing pipeline. The Supervisor maintains a registry of workload-specific re-dispatch handlers. For each workload type (e.g., DNA, Fractals, GrayScott), a callback is registered that allows the system to re-process requests transparently.

This is implemented using asynchronous re-handling threads to avoid blocking the scaling loop. Each request is reprocessed by invoking the same handler logic as an incoming request, ensuring consistent cost estimation and scheduling decisions.

From the client perspective, requests are never explicitly failed due to worker termination. From the system perspective, execution is resumed on another available worker or fallback mechanism. Finally, from the scheduler perspective, re-dispatched requests are treated as fresh requests with updated system state awareness.

9.4 Fallback Execution Path

In addition to worker-level fault tolerance, the system provides higher-level resilience through fallback execution strategies:

- If no suitable EC2 worker is available, requests may be executed on AWS Lambda (when within cost constraints).
- If both EC2 and Lambda are unavailable, requests are temporarily delayed and retried via re-dispatch.

In the re-dispatch path, the load balancer enters a bounded retry phase, periodically attempting to obtain a worker from the Supervisor.

The retry mechanism performs up to 300 attempts with a delay of 100 ms between attempts, corresponding to a maximum waiting period of approximately 30 seconds. This allows the system to tolerate temporary capacity shortages caused by worker startup delays, ongoing request execution, or auto-scaling events.

If a worker becomes available during this interval, the request proceeds normally. Otherwise, the request is rejected with an appropriate error response. By delaying failure during transient overload conditions, this mechanism improves system robustness while avoiding indefinite request blocking.

This waiting period complements the auto-scaling mechanism. Newly launched instances may require several seconds before becoming available, and delaying failure allows requests to benefit from capacity that is already being provisioned rather than being rejected immediately.

9.5 Design Rationale

Compared to the checkpoint architecture, where fault tolerance was largely implicit in AWS-managed infrastructure, the final design introduces explicit request-level recovery logic. This provides finer-grained control over execution continuity and avoids unnecessary request loss during auto-scaling transitions, instance termination events or transient worker failures.

Overall, the system ensures that request execution is resilient to both infrastructure-level and application-level failures, while preserving correctness of scheduling decisions through re-integration into the cost-aware load balancing pipeline.

Input: worker failure or termination event
Output: preserved execution of all in-flight requests

```

1: Move worker  $w$  to TERMINATING pool
2: if  $w$  fails health check then
3:   Move  $w$  to NON_RESPONSIVE pool
4:   Wait recovery grace period
5:   if  $w$  recovers then
6:     Move  $w$  back to WORKING pool
7:   return
8: else
9:   Mark  $w$  for termination
10: end if
11: end if
    ▷ ————— Drain in-flight requests —————
12: if  $w$  has pending requests then
13:    $R \leftarrow w.\text{drainPendingExchanges}()$ 
14:   for all  $(\text{exchange}, \text{cost}) \in R$  do
15:      $\text{type} \leftarrow \text{extractWorkloadType}(\text{exchange})$ 
16:      $\text{handler} \leftarrow \text{rehandleRegistry}[\text{type}]$ 
17:     if  $\text{handler} \neq \text{null}$  then
18:       re-invoke  $\text{handler}(\text{exchange}, \text{cost})$ 
19:     end if
20:   end for
21: end if
    ▷ ————— Fallback execution —————
22: for all new or re-dispatched request  $r$  do
23:    $\text{cost} \leftarrow \text{estimateCost}(r)$ 
24:   if EC2 worker available then
25:     assign  $(r, \text{worker})$ 
26:   else if  $\text{cost} \leq \text{LAMBDA\_MAX}$  then
27:     execute  $r$  on AWS Lambda
28:   else
29:     for  $i \leftarrow 1$  to 300 do
30:       wait 100 ms
31:       if EC2 worker becomes available then
32:         assign  $(r, \text{worker})$ 
33:         break
34:       end if
35:     end for
36:     if no worker found then
37:       return 503 Service Unavailable
38:     end if
39:   end if
40: end for

```

10 DESIGN EVOLUTION AND ALTERNATIVES CONSIDERED

10.1 Checkpoint vs Final Design

The checkpoint architecture relied more heavily on AWS-managed components and DynamoDB queries for historical cost lookup. While simple, it introduced latency and reduced control over scheduling decisions.

The final architecture replaces synchronous DynamoDB lookups with local caching of bucketed cost estimates, periodic asynchronous refresh, and immediate fallback cost estimation on miss.

10.2 Rejected Approaches

We considered the following alternatives:

1. **Always query DynamoDB on miss:** Rejected due to high latency and cost.
2. **Fully stateless estimation model:** Rejected due to loss of historical accuracy.
3. **Machine learning regression model:** Rejected due to training overhead and lack of interpretability under strict runtime constraints.

11.1 Cold Start Effects

When a workload bucket has never been observed before, the system falls back to a heuristic cost estimation. These estimates are naturally less accurate than historical measurements and may temporarily lead to suboptimal scheduling decisions. A consequence is the quality of the scheduling policy improves over time as more requests are processed and additional metrics become available.

11.2 Partially Reactive Auto Scaling

The auto-scaling mechanism is primarily reactive, as scale-up decisions are triggered by observed CPU utilization and worker availability. However, the implementation incorporates limited predictive behavior. Pending instances are immediately accounted for during scaling decisions, preventing repeated launches while new workers are still initializing. Furthermore, scale-down decisions estimate the projected average CPU utilization after worker removal and only proceed when the remaining capacity is expected to remain below the configured threshold. Despite these anticipatory elements, scaling remains fundamentally driven by current system conditions rather than future workload forecasts.

11.3 Single Load Balancer Instance

The architecture currently employs a single load balancer instance. While worker nodes can scale horizontally, the load balancer itself remains a single point of failure.

11.4 Metric Collection Overhead

The instruction-level instrumentation used to collect execution metrics introduces additional runtime overhead. Although this overhead is relatively small compared to the execution time of the scientific workloads considered in this project, it is not negligible. For extremely short requests, instrumentation costs may represent a noticeable fraction of total execution time.

12 IMPLEMENTATION NOTES

During the writing of this report, a minor bug was identified in the worker selection logic. The load admission condition uses " $\text{worker.getLoad()} + \text{cost} < \text{MAX_LOAD_THRESHOLD}$ " instead of the intended " $\text{worker.getLoad()} + \text{cost} \leq \text{MAX_LOAD_THRESHOLD}$ ". As a result, requests whose combined load exactly matches the threshold may be rejected.

Secondly, although AWS Lambda functions were successfully created and deployed for the system, the deployment process did not include an explicit binding between workload types and their corresponding Lambda function names. As a result, the load balancer was designed to rely on an environment variable ($\text{LAMBDA_FUNCTION_NAME}$) to determine the target function at invocation time, but this variable was never defined or injected per workload. This led to an incomplete invocation contract: Lambda functions existed but were correctly bound, the system lacked a deterministic mapping from workload type to specific Lambda function identifiers during execution. A complete deployment setup would define explicit workload-to-Lambda mappings.

Finally, fault tolerance logic was missing when auto-scaler removes workers marked to terminate (in a TERMINATING POOL since they have crashed). For idle workers we ensure fault tolerance to make sure no requests are missing.

13 CONCLUSION

The final system implements a fully distributed, cost-aware load balancing architecture capable of adapting to heterogeneous workloads. By integrating instrumentation, bucketed historical caching, and asynchronous data refresh mechanisms, the system achieves improved scalability and responsiveness. Overall, the project demonstrates how combining runtime metrics with lightweight predictive caching can significantly improve distributed workload scheduling under real-world cloud constraints.